Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение высшего образования “Омский государственный технический университет”
Кафедра “Автоматизированные системы обработки информации и управления”

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 1

по дисциплине «Кроссплатформенная разработка»
студента Колохматова Андрея Максимовича группы ПИН-252т

Пояснительная записка
Шифр работы От-43-ПИН-252т-1 ЛР
Специальность 09.03.03 Программная инженерия

Преподаватель М.А. Мищенко
Студент А.М. Колохматов

Омск 2026

РЕФЕРАТ

Объектом выполненной работы является кроссплатформенное настольное приложение «Рэп-каталог», разработанное на Kotlin с использованием Compose Multiplatform.

Целью данной работы является построение каркаса приложения: навигации между экранами, экрана-списка на моковых данных, переключения темы и локализации интерфейса.

В работе применены архитектурный подход с разделением на слои domain, data и ui, однонаправленный поток данных (состояние экрана и намерения пользователя), библиотека Navigation 3 и механизм строковых ресурсов Compose Multiplatform.

Степень внедрения — для учебных целей.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4
1 ПОСТАНОВКА ЗАДАЧИ 6
2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ 7
3 РЕАЛИЗАЦИЯ 8
4 ПРОВЕРКА РАБОТЫ 11
ЗАКЛЮЧЕНИЕ 16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 17

ВВЕДЕНИЕ

Кроссплатформенная разработка позволяет описать интерфейс и логику приложения один раз и собрать их под несколько платформ. Технология Compose Multiplatform расширяет декларативный подход Jetpack Compose на настольные системы, веб и iOS: общий код лежит в одном модуле, а платформенные точки входа только запускают его.

Предметной областью приложения выбран каталог хип-хоп исполнителей. Источник данных — открытый программный интерфейс MusicBrainz, который отдаёт сведения об артистах, их жанрах, дискографии и связанных исполнителях. На первой вехе курса сеть не используется: приложение работает на моковых данных, повторяющих форму ответа сервера, поэтому на следующей вехе моки заменяются сетевым источником без изменения остального кода.

Цель работы — разработать каркас кроссплатформенного приложения, то есть такую основу, которая не рассыпается при обращении с ней: при переключении темы, смене языка, переходе на экран детали и возврате назад.

Для достижения цели поставлены следующие задачи:

— создать проект Kotlin Multiplatform из мастера JetBrains с таргетами Android, iOS, настольная система и веб;

— разложить код по слоям domain, data и ui так, чтобы зависимости сходились к слою предметной области;

— подготовить моковый набор данных не менее чем из двадцати артистов по форме программного интерфейса;

— реализовать экран-список в виде сетки карточек и экран детали артиста со справкой, дискографией и переходами к связанным исполнителям;

— добавить поиск по имени исполнителя: подстрока сужает сетку, запрос сохраняется при возврате с детали, при отсутствии совпадений выводится сообщение «Ничего не найдено»;

— организовать навигацию на библиотеке Navigation 3, где маршрут является объектом, а не строкой;

— вынести цвета в тему и реализовать переключение светлого и тёмного оформления;

— вынести все подписи интерфейса в строковые ресурсы и добавить вторую локаль;

— проверить собираемость всех таргетов и включить отчёт компилятора Compose.

1 ПОСТАНОВКА ЗАДАЧИ

Приложение представляет собой каталог хип-хоп исполнителей и состоит из двух экранов: списка артистов в виде сетки и детали артиста. Список показывает фотографию, имя и ведущий жанр каждого артиста. Деталь содержит сведения, которых нет в списке: короткую справку об исполнителе, страну и место рождения, годы жизни или существования группы, псевдонимы, дискографию, теги с весом и связанных артистов. Над сеткой расположено поле поиска по имени исполнителя: ввод подстроки сужает сетку, а при отсутствии совпадений вместо неё выводится сообщение «Ничего не найдено».

Источником данных на последующих вехах выступает программный интерфейс MusicBrainz по адресу https://musicbrainz.org/ws/2/. Он не требует ключа, поддерживает постраничную выдачу параметрами limit и offset и содержит сведения о сотнях тысяч исполнителей. Ограничение источника — один запрос в секунду с одного адреса и обязательный заголовок User-Agent с контактными данными.

Главные требования к каркасу: маршрут навигации является объектом, а не строкой; возврат назад выглядит как возврат; все подписи вынесены в строковые ресурсы на двух языках; цвета берутся из темы; экраны рисуются из состояния; есть поиск по имени исполнителя; включён отчёт компилятора Compose.

2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ

Код приложения разделён на три слоя. Слой domain содержит модели предметной области (Artist, Genre) и описание репозитория в виде интерфейса. Слой data содержит моковые данные и реализацию репозитория. Слой ui содержит экраны, переиспользуемые компоненты, тему, навигацию и модели представления. Все зависимости направлены к слою domain: слой ui пользуется его моделями, слой data преобразует полученные данные в эти же модели.

Такое разделение решает практическую задачу: на следующей вехе моковый репозиторий заменяется сетевым, и вся правка происходит внутри папки data, а экраны и модели предметной области остаются неизменными.

Экраны лежат в пакетах ui/list и ui/detail: в каждом находятся вёрстка экрана, его состояние, намерения, ViewModel и фабрика. Рядом в пакете ui расположены тема, локаль, навигация и общие компоненты интерфейса, а фотографии артистов, иконки и подписи хранятся в ресурсах composeResources.

Логика каждого экрана вынесена в отдельный класс ViewModel. Внутри него состояние хранится в изменяемом потоке, а наружу отдаётся поток только для чтения, поэтому изменить состояние извне невозможно. Действия пользователя приходят в единственный метод обработки намерений.

Зависимости уровня приложения — репозиторий, навигатор и фабрики ViewModel — создаются в корневой функции приложения и передаются вниз. Фабрики обязательны: в общем коде нет отражения, с помощью которого Android создаёт ViewModel без аргументов, и на iOS и в вебе такого механизма не существует.

3 РЕАЛИЗАЦИЯ

3.1 Модели предметной области и репозиторий

Модели повторяют форму ответа программного интерфейса, включая названия полей и типы: идентификатор артиста — строка стандарта UUID (MBID).

3.2 Моковые данные

Моки содержат тридцать три записи. Данные снимались с программного интерфейса MusicBrainz напрямую, поэтому имена, страны, даты, теги и дискография являются настоящими: 410 релиз-групп суммарно.

Поскольку сети на этой вехе нет, фотографии артистов хранятся в ресурсах приложения. Снимки взяты с Викисклада по ссылке между MBID и Викиданными; у каждого — указан автор и лицензия в docs/CREDITS.md. Запасной вариант для записи без свободного снимка — плитка с первой буквой имени на градиенте цвета жанра.

3.3 Состояние экрана и намерения

Экран не обрабатывает нажатия самостоятельно. Ему передаются состояние и функция обработки намерений, а решение принимает ViewModel. Такой порядок нужен не только для стройности кода: если передать в экран целиком ViewModel, компилятор Compose признает её нестабильным типом, начнёт сравнивать по ссылке и перестанет пропускать перерисовку экрана.

Список выводится сеткой из трёх колонок (LazyVerticalGrid), каждому элементу задан ключ по MBID. Ключ обеспечивает сохранение позиции прокрутки при изменениях списка, поскольку память привязывается к ключу, а не к номеру позиции.

Поиск по имени исполнителя устроен по той же схеме. Ввод в поле поиска превращается в намерение QueryChanged, а ViewModel кладёт в состояние и сам запрос, и отфильтрованный список. Сравнение идёт без учёта регистра, пробелы по краям запроса отбрасываются. Запрос хранится в ViewModel, а не в локальном состоянии экрана, поэтому он переживает переход на деталь и возврат: ViewModel списка привязана к хранилищу уровня окна и при открытии детали не уничтожается. Если совпадений нет, вместо сетки

выводится сообщение «Ничего не найдено», а счётчик над сеткой во время поиска показывает число найденных записей, а не размер каталога.

3.4 Навигация

В библиотеке Navigation 3 стека маршрутов как отдельной сущности нет: стек — это обычный список объектов. Маршруты описаны sealed-интерфейсом, а аргумент лежит полем объекта, поэтому опечатку в аргументе ловит компилятор, а не выполнение программы.

Хост навигации описывает переходы и сопоставляет маршрутам экраны. Все три спецификации перехода заданы явно: прямая, обратная и предиктивная. Это не оформление, а условие работы — значения по умолчанию для непрямых переходов вне платформы Android являются заглушками и приводят к ошибке во время выполнения.

Классу ViewModel детали задан ключ, составленный из MBID артиста. Без ключа обращение к ViewModel возвращает один экземпляр на весь хост навигации, и открытая повторно деталь показала бы предыдущую запись.

3.5 Тема

Тема представляет собой набор токенов оформления, который провайдер кладёт в неявный контекст композиции. Чтение токена является чтением состояния, поэтому при смене темы перерисовываются именно те элементы, которые её прочитали. Цвет, записанный числом прямо в коде элемента, в переключении темы не участвует, так как мимо контекста подписка на изменение не возникает. По этой причине все цвета объявлены в одном файле.

Исключением из правила является цвет жанра: он несёт смысл данных, поэтому не меняется вместе с темой. Литералы этого цвета всё равно объявлены в отдельном файле, а наружу отдаётся функция, выбирающая цвет по жанру. Переключатель темы размещён в заголовке приложения и хранит своё значение в памяти сеанса: сохранение настроек на диск относится к третьей вехе курса.

3.6 Локализация

Все подписи интерфейса вынесены в строковые ресурсы общего модуля. По ним создаётся класс доступа, поэтому в коде стоит не строковое имя, а сгенерированный идентификатор, и опечатка не компилируется. Квалификатор

локали разрешается во время выполнения: один и тот же идентификатор даёт разный текст.

Паритет ключей между локалями проверяется сценарием tools/check-strings.py: ключ, забытый в одной локали, при переключении языка оставил бы на экране чужой язык, и глазами такая ошибка находится не всегда. Локализуется интерфейс, а не данные каталога: имена артистов и названия альбомов остаются на языке источника.

Отдельно локализована короткая справка об исполнителе: помимо английского текста (первый абзац статьи английской Википедии) у неё есть русская версия — это не машинный перевод, а первый абзац отдельной статьи русской Википедии со своими авторами. Программный интерфейс MusicBrainz такого текста не отдаёт: его поле annotation служебное, для редакторов базы, а не для читателя.

4 ПРОВЕРКА РАБОТЫ

Приложение собирается и запускается на всех таргетах проекта: настольная система, Android и веб.

Ведущим таргетом первой вехи выбрана настольная система, зачётными таргетами курса — настольная система и Android. Сборка фреймворка для iOS также выполняется, хотя платформенный код для неё сводится к минимальным expect/actual-реализациям.

Проверка работы выполнялась вручную: прокрутка списка, переход на деталь и возврат, переход к связанному артисту, переключение темы и языка, поиск по подстроке имени, возврат с детали с сохранением запроса и поиск несуществующего имени. Паритет ключей локалей проверен сценарием check-strings.py: расхождений нет, проверен 51 ключ.

Отчёт компилятора Compose включён в сборке общего модуля (shared/build.gradle.kts) и выводится в каталог shared/build/compose_compiler. Отчёт пишется во время компиляции, поэтому смотреть его нужно после сборки, например после ./gradlew :shared:compileKotlinJvm.

В первой версии работы файл отчёта выглядел пустым, и это было ошибочно списано на версию Kotlin. Настоящая причина — имя модуля. По умолчанию оно равно RapCatalog:shared, и компилятор создаёт файл RapCatalog:shared-classes.txt. В файловой системе NTFS двоеточие в имени отделяет альтернативный поток данных, поэтому в проводнике появлялся пустой файл RapCatalog, а содержимое отчёта лежало в его скрытых потоках. Модулю задано имя shared (параметр moduleName для таргетов JVM и Android), и отчёт стал записываться в обычные файлы. На macOS и Linux проблемы не было бы вовсе.

По отчёту все 25 перезапускаемых composable-функций модуля являются пропускаемыми (restartable skippable), в том числе ArtistListScreen и ArtistDetailScreen. Классы ArtistListViewModel и ArtistDetailViewModel помечены как нестабильные (unstable), но в функции экранов они не передаются: экраны получают состояние и обработчик намерений. Классы состояния помечены как runtime, потому что содержат List, а стабильность интерфейса List компилятор доказать не может; в режиме строгого пропуска

(strong skipping) такие параметры сравниваются по ссылке, и пропуск перерисовки сохраняется.

Внешний вид приложения приведён на рисунках 1–4, работа поиска — на рисунках 5–7.

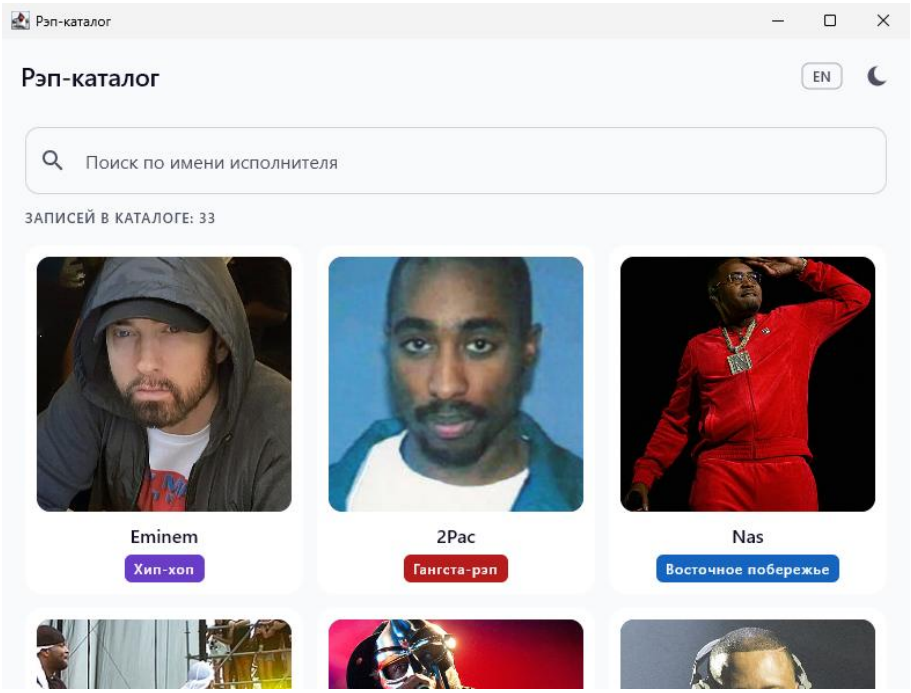


Рисунок 1 — Экран списка артистов, светлая тема

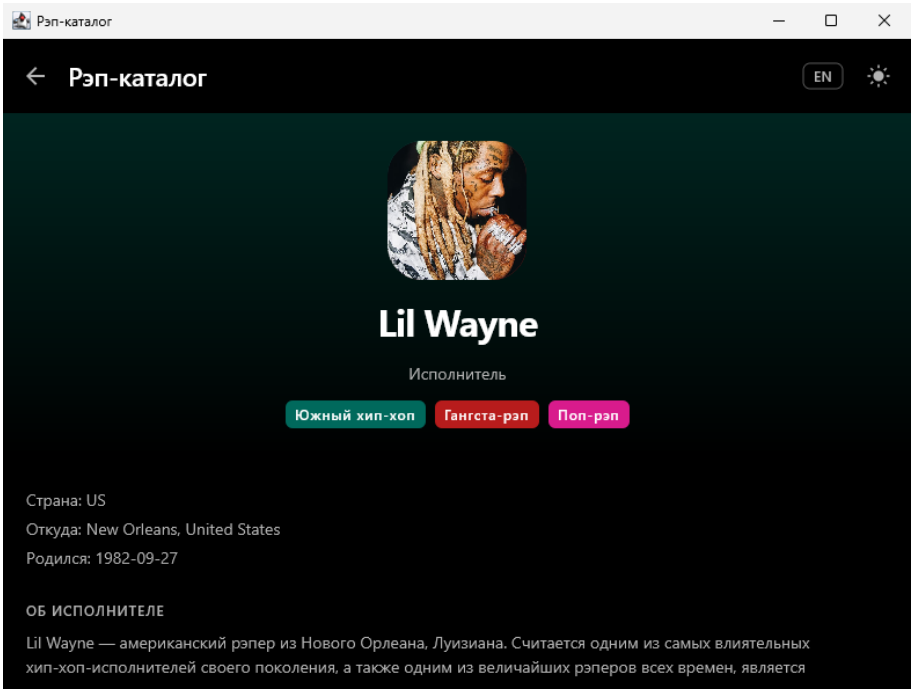


Рисунок 2 — Экран детали артиста, тёмная тема

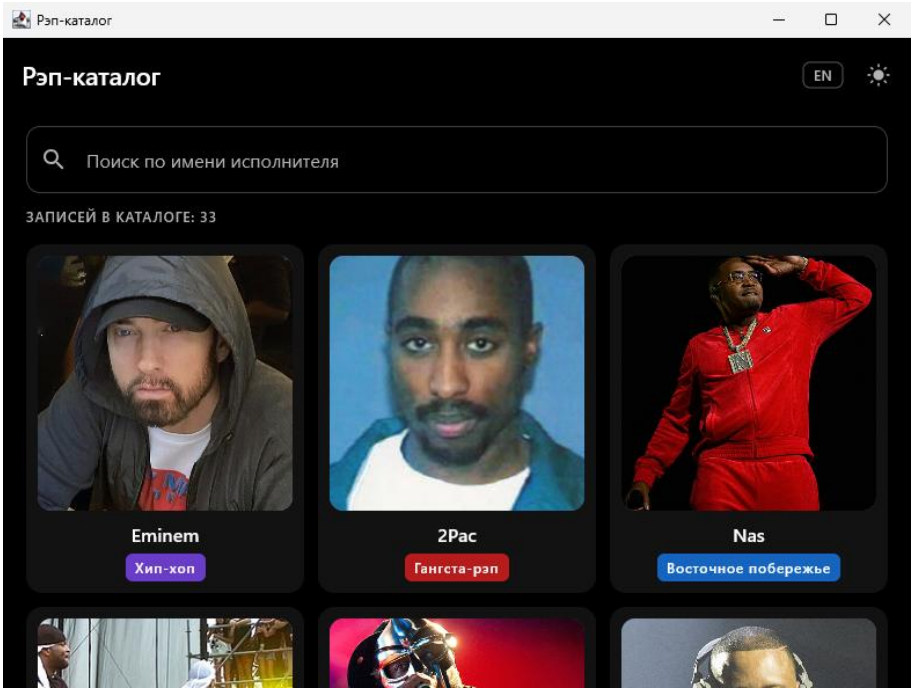


Рисунок 3 — Экран списка артистов, тёмная тема

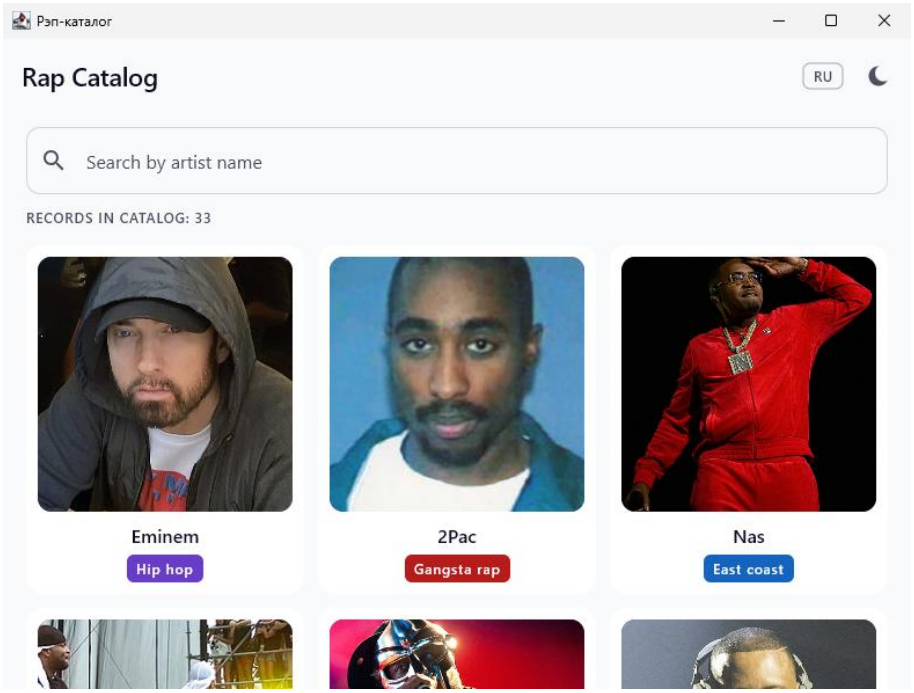


Рисунок 4 — Интерфейс на английском языке

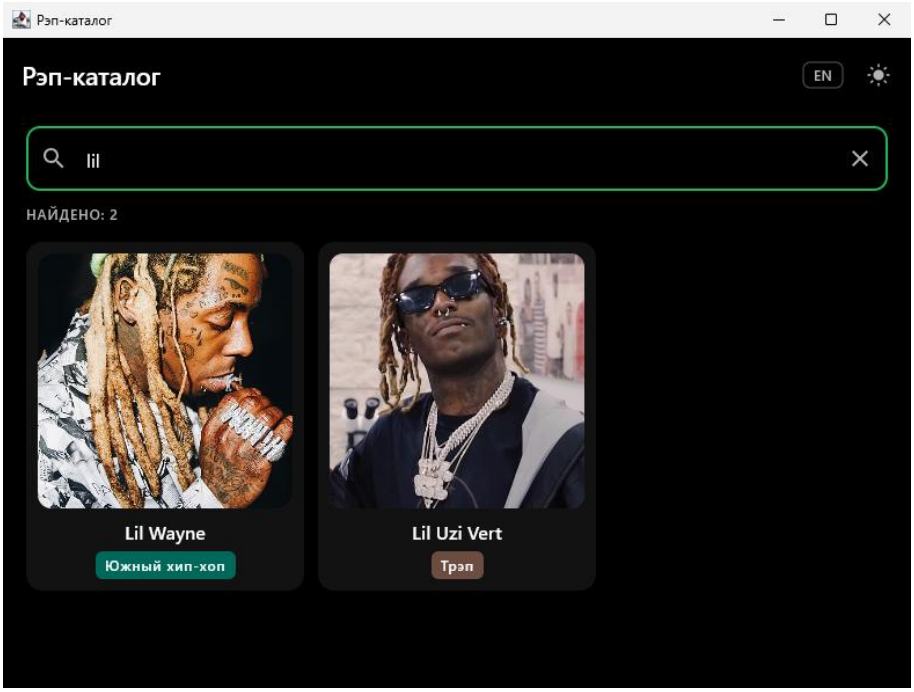


Рисунок 5 — Поиск: подстрока «lil» сужает сетку

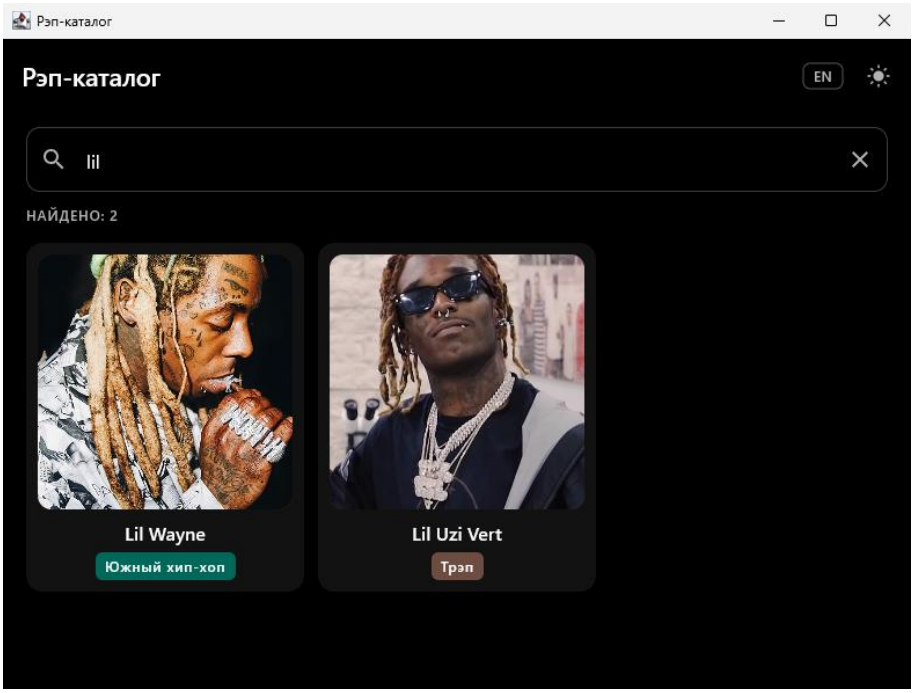


Рисунок 6 — Возврат с детали: запрос сохранён

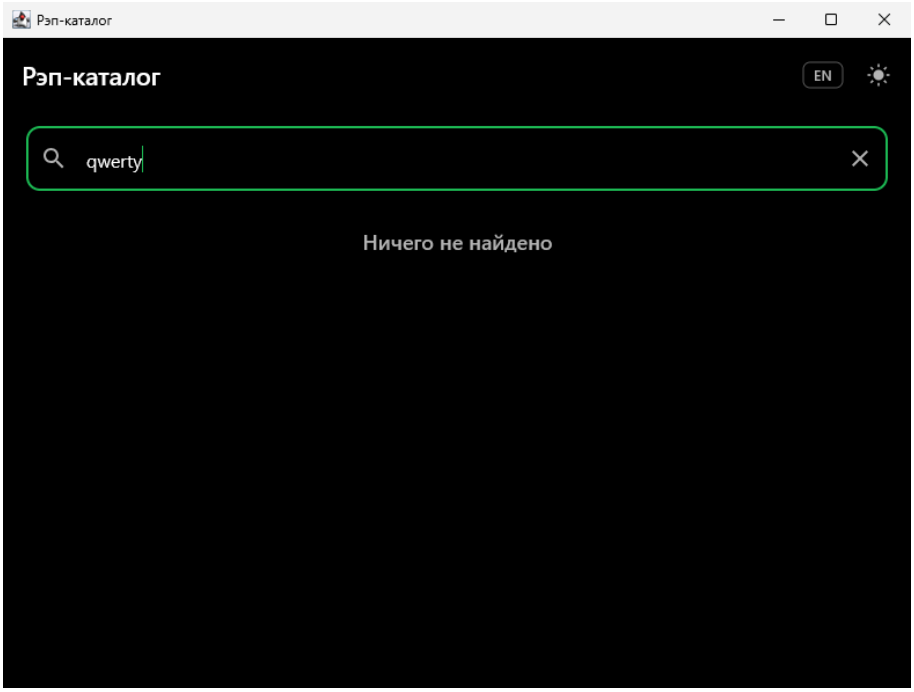


Рисунок 7 — Поиск без совпадений

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы поставленная цель достигнута: разработан каркас кроссплатформенного приложения «Рэп-каталог» на Compose Multiplatform. Все поставленные задачи выполнены.

Код разложен по слоям domain, data и ui, зависимости направлены к слою предметной области, а моковый набор из тридцати трёх артистов повторяет форму программного интерфейса MusicBrainz. Благодаря этому замена моков на сетевой источник на следующей вехе затронет только слой data.

Реализованы экран списка в виде сетки с поиском по имени исполнителя и экран детали, навигация между ними на библиотеке Navigation 3, где маршрут является объектом, а стек маршрутов — обычным списком. Экраны получают состояние и обработчик намерений, а не класс ViewModel целиком. Цвета вынесены в тему, переключение светлого и тёмного оформления работает, подписи интерфейса вынесены в строковые ресурсы и переведены на второй язык, включая двуязычную справку об исполнителе; паритет ключей локалей проверяется отдельным сценарием.

Отдельный вывод сделан по отчёту компилятора Compose: пустой файл отчёта оказался следствием двоеточия в имени модуля на файловой системе NTFS, а не ошибкой компилятора. После исправления отчёт подтвердил, что все перезапускаемые функции модуля пропускаемые. Практическая ценность работы состоит в полученном опыте разделения приложения на слои и в понимании механизмов декларативного интерфейса: подписки на состояние при чтении, пропуска перерисовки и разрешения ресурсов по квалификатору локали. Полученный каркас служит основой для следующей вехи курса, на которой будут добавлены работа с сетью, постраничная загрузка, состояния экрана и кэширование данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Compose Multiplatform: официальная документация [Электронный ресурс]. — URL: https://www.jetbrains.com/help/kotlin-multiplatform-dev/compose-multiplatform.html

2. Navigation 3: официальная документация [Электронный ресурс]. — URL: https://developer.android.com/guide/navigation/navigation-3

3. Material Design 3: цветовые схемы и токены [Электронный ресурс]. — URL: https://m3.material.io/styles/color/system

4. MusicBrainz: описание программного интерфейса [Электронный ресурс]. — URL: https://musicbrainz.org/doc/MusicBrainz_API

5. Compose compiler reports [Электронный ресурс]. — URL: https://developer.android.com/develop/ui/compose/performance/stability/diagnose